Lab 11

Files, Streams, and Object Serialization

Objectives: ***At the end of this lab session the student should be able to:***

- Read and write text files.
- Work with object serialization.

- All data in and out of a computer is treated by the system as a **stream** of data. Also, the term **stream** refers to ordered data that is read from or written to a file. Java views each file as a sequential stream of bytes.
- File streams can be used to input and output data as either **characters** or **bytes**.
- Streams that input and output characters to files are known as **character-based streams,** storing data as a **sequence of characters.** Streams that input and output bytes to files are known as **byte-based streams,** storing data in its **binary format.**
- Files that are created using byte-based streams are referred to as **binary files**. Files that are created using character-based streams are referred to as **text files**.
- Object serialization is the ability to read an entire object from or write an entire object to a file.
- A *serialized* object is represented as a sequence of bytes that includes the object's data and its type information.
- After a *serialized* object has been written into a file, it can be read from the file and *deserialized* to recreate the object in memory.
- Objects of classes that implement interface **Serializable** can be serialized and de-serialized with **ObjectOutputStreams** and **ObjectInputStreams**.

Part 1: Problem Solving Exercises (Time: 60 minutes)

Exercise 1: Text Files (Working with **Formatter** class)

1. Add to the project of exercise 1 the following class. Then run the app and type any text that you want to save. When you finish typing, enter `<Ctrl+Z>`. Then open the file to see what you typed.

```
package files;
import java.util.Formatter;
import java.util.Scanner;
import java.io.File;
import java.io.IOException;
public class FormatterDemo {
  public static void main(String[] args) {
        Formatter out=null;
        Scanner kb = new Scanner(System.in);

        try {// out1 = new Formatter("MyOutFile.txt");
              out = new Formatter(new File("MyOutFile.txt"));
        } catch (IOException ioe) {
              System.out.println("File is not opened. Exception occurred..\n"+ioe);
        }
```

```
            if (out != null) {
                System.out.println("Enter data to be saved. When done enter <Ctrl+Z>");
                String line;
                while (kb.hasNext()) {
                    line = kb.nextLine();
                    out.format("%s%n", line);
                }
                out.close();
            }
            kb.close();
        }

    }
```

1. Run the program and enter your name and then finish input. Then open the file and see what you typed. Close the file, run the program once more and then type "CMPS251 Lab" and then terminate your program. Does it replace the previous content or is it added at the end of the file?
2. How can you add new content to an existing file without replacing the old content? _____

Exercise 2: Text Files (Working with **Formatter** and **FileWriter** classes)

3. Add to the project of exercise 1 the following class. Then run the app and type any text that you want to save. When you finish typing, enter `<Ctrl+Z>`. Then open the file to see what you typed.

```
package files;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Formatter;
import java.util.Scanner;
public class FileWriterDemo {
    public static void main(String[] args) {
        Formatter out=null;
        FileWriter fileWriter;
        Scanner kb = new Scanner(System.in);
        try {
           fileWriter = new FileWriter("MyOutFile.txt", true);
           out = new Formatter(fileWriter);
        } catch (IOException ioe) {
           System.out.println("File is not opened. Exception occurred..\n"+ ioe);
        }
        if (out != null) {
           System.out.println("Enter data to be saved. When done enter<Ctrl+Z>");
           String line;
           while (kb.hasNext()) {
                        line = kb.nextLine();
                        out.format("%s\n", line);
           }
           out.close();
        }
        kb.close();
    }
}
```

4. Run the program once more and enter your name and then finish input. Then open the file and see what you typed. Does it replace the previous content, or it is added at the end of the file?

   _____
5. What is the purpose of the **true** keyword when added as a second argument in the FileWriter Constructor? _____
6. What happens to the file content if you do not provide this second argument? _____

Exercise 3: Object Serialization

1. Add to the project Lab10 the following two classes. Then run the app and view the output.

```
package files;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.EOFException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
public class ObjectSerialization {
  public static void main(String[] args) {
        ObjectOutputStream out;
        ObjectInputStream in;
        try {
              out = new ObjectOutputStream(new FileOutputStream("persons.dat"));
              Person p1, p2, p;
              p1 = new Person("Ahmed");
              p2 = new Person("Hind");
              // Saving objects
              out.writeObject(p1);
              out.writeObject(p2);
              out.close();

              // Reading objects
              in = new ObjectInputStream(new FileInputStream("persons.dat"));
              Object obj;
              while ((obj = in.readObject()) != null) {
                    p = (Person) obj;
                    System.out.println(p.getName());
              }
        }catch (EOFException eof){
              System.out.println("Reached end of file.");
        } catch(IOException | ClassNotFoundException exception) {
              System.out.println(exception.getStackTrace());
        }
  }
}
```

```
  package files;
  public class Person {
        private String name;
        public String getName() {
              return name;
        }
        public void setName(String name) {
              this.name = name;
        }
        public Person(String name) {
              this.name = name;
        }
  }
```

2. Did you obtain the names of the two created person objects? Were they saved? If not why?

   _____

3. Let the Person class implement the Serializable interface and then run the application again. Compare the output with that of the previous run. What is your conclusion?

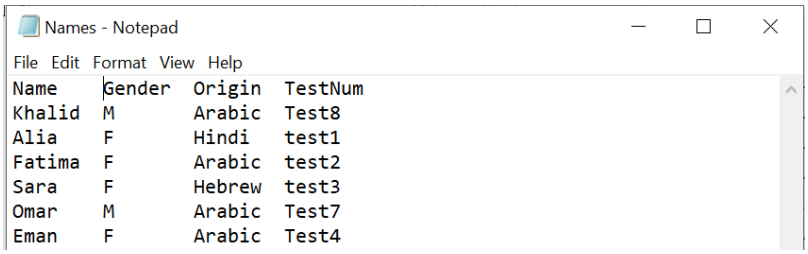
   _____

Part 2: Practice Exercises (Time: 90 minutes)

Exercise 4: read from file using Scanner

Given the **Names.txt** text file that contains names of girls and boys, their origins and test number (obtain it from the course website – a screenshot showing the first few lines is given below), write

an application that opens the file and list only girls' names of Arabic origin. Your listing should only show the name, gender, and test number. Do not show the origin as it is that same for all the required output. Properly format your output. Let your program send a copy of the output to a text file called **ArabicNames.txt**.



Exercise 5

1. Modify the Person class of Exercise 3 by adding two more attributes: id (int) and salary (double). Add setters and getters and modify the constructor to have three parameters to initialize all attributes.
2. Write a new application that asks the user to enter a Person's information, create a Person's instance and then save it in a file of objects named **employees.dat**. The program continues to accept data and sends them as objects to the above file. The program terminates when the user enters a negative **id**.
3. Write another application that opens the employee.dat file and loads its contents in an ArrayList of type Person.
4. Write a method called *getPerson(int searchId)* that searches the above array list for a Person with id equals to the searchId. If found the function returns that Person instance, if not found the method returns null.
5. Test the getPerson() method. Print the salary for the test person if found.

Additional Practice Problems

Exercise 6:

1. Write Java program to read student's data that includes name (string), and three grades (double). Your program should ask user at the beginning how many students he wants to enter, then keep reading this data and save them to a file called "Students.txt"
2. Write a program to read students' data from the file and display student's name and average .

Exercise 7:

1. Create a text file (Employee.txt) that contains data about employees as following

234	Sara	4500.00
987	Khaled	7345.12
997	Nasser	1267.00
523	Noora	8230.00

2. Create a class Employee that has three data: id(int), name(String), salary (double), add constructor, and setter/getter methods.
3. Write a program that reads data from the file "Employee.txt" to an ArrayList of Employee, then save this ArrayList to a binary fine called "Employee. dat" as **one object**.
4. Write another program to retrieve the data saved in the file "Employee.dat", and display employee records on the screen.